

Ingeniería del Software 3

Clase 3 · Planificación y trazabilidad

UCC · Ingeniería en Sistemas · 2026 · Ing. Ariel Schwindt

Lo que vas a ver en este video

- El práctico de hoy se hace sobre el repositorio del TP2 — no hace falta tocar código de la app

Bloque	De qué se trata
La teoría	Plan + código juntos · jerarquía · historias e INVEST · criterios de aceptación · límite de trabajo en progreso · trazabilidad
Las herramientas	GitHub Projects (riel canónico) y Azure Boards (riel alternativo)
El práctico	Qué tenés que entregar — y qué NO
Y después	El paso a paso completo, en el video del TP3

Por qué el plan y el código tienen que vivir JUNTOS

El mundo separado

- Plan en un Excel/Trello; código en el repo
- La conexión vive en la memoria de la gente
- El plan dice una cosa, el código otra
- "¿Esto ya está hecho?" → preguntar en el pasillo

La plataforma DevOps

- El plan vive ALREDEDOR del repo
- Un PR CIERRA la historia que implementa
- La historia muestra qué commits la materializaron
- "¿Esto ya está hecho?" → datos, no memoria

- GitHub, Azure DevOps y GitLab integran gestión del trabajo + código en un solo sistema con datos ENLAZADOS
- DORA mide commit→prod; el plan enlazado te deja medir la cadena COMPLETA: decisión→prod

La jerarquía de trabajo: del objetivo a la tarea

Tipo	Qué representa	Tamaño	Ejemplo (de esta materia)
Épica	Objetivo grande que agrupa valor	Semanas+	"Pipeline DevOps completo para mi app"
Historia	Incremento de valor OBSERVABLE, en lenguaje de usuario	Días	"Como dev quiero que cada PR corra los tests..."
Tarea	Trabajo técnico concreto dentro de una historia	Horas	"Escribir el workflow de build"
Bug (al costado)	Defecto de algo YA entregado — no es un nivel: no cuelga de nadie	—	"El compose no espera a la BD"

- **La jerarquía no es burocracia: es ZOOM**
- El cliente razona a nivel épica-historia (¿qué valor entregamos?); el equipo ejecuta a nivel tarea (¿qué hago hoy?)
- La trazabilidad entre niveles permite navegar del 'por qué' al 'cómo' — y viceversa

Historias de usuario: el formato donde cada parte trabaja

"Como **[rol]** quiero **[capacidad]** para **[beneficio]**"

Parte	Qué te obliga a saber
Rol	PARA QUIÉN es — si no sabés quién lo quiere, ¿por qué lo hacés?
Capacidad	El QUÉ, sin fijar el CÓMO (el cómo es del equipo)
Beneficio	El POR QUÉ — si no podés escribir el 'para', quizás no haya que hacerla

No es un mantra para recitar: es un checklist de tres preguntas disfrazado de oración.

INVEST: seis chequeos (y los anti-patrones de siempre)

Letra	Chequeo	La trampa habitual
I — Independiente	Se puede hacer sola	Cadenas de historias bloqueadas entre sí
N — Negociable	Es una conversación, no un contrato	Especificada al milímetro desde el día 1
V — Valiosa	Alguien la quiere	Historias 'técnicas' sin beneficiario
E — Estimable	Se entiende lo suficiente	'Investigar X' sin alcance
S — Small	Cabe en DÍAS, no semanas	La historia-elefante que nunca termina
T — Testeable	Se puede VERIFICAR	'Mejorar la performance' (¿cuánto?)

- **Anti-patrón bonus: la tarea disfrazada** — "Como dev quiero crear la tabla usuarios" (nadie 'quiere' una tabla: es una TAREA)
- Las dos que más se violan: Small y Testeable — y Testeable tiene su propia solución: el próximo slide

Criterios de aceptación: cuándo la historia está hecha

```
Historia: "Como dev quiero que cada PR corra los tests automáticamente  
para detectar regresiones antes del merge"
```

Criterios de aceptación

- [] El workflow corre en cada PR a main
- [] Un test que falla bloquea el merge
- [] El reporte de tests queda publicado como artefacto
- [] Badge de estado visible en el README

- La lista concreta y VERIFICABLE de condiciones que hacen que la historia esté 'hecha'
- Son de ESA historia. La Definition of Done es la lista que vale para TODAS (revisada, con tests, mergeada) — no las confundas
- Una historia sin criterios de aceptación no se puede demostrar terminada — en esta materia, no existe

Límite de trabajo en progreso: empezar menos, terminar más

10 cosas 'en progreso':



→ valor entregado: CERO (nada terminado)

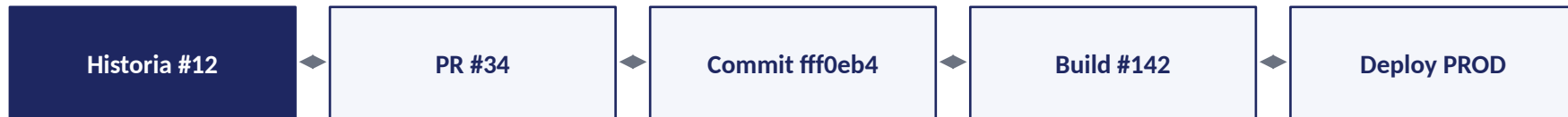
3 cosas con el límite puesto:



→ valor entregado: 3 (y las siguientes en camino)

- El trabajo empezado y no terminado no es productividad: es INVENTARIO
- Cada cosa 'en progreso' extra = más cambio de contexto, más ramas viejas, más merge hell (¿te suena de la clase 1?)
- El límite de trabajo en progreso (WIP limit, por work in progress) = la traducción operativa de 'cambios chicos y frecuentes'
-  La herramienta NO bloquea: pone el contador en rojo y te deja pasar — el límite es un acuerdo del equipo
- Número de arranque: cantidad de personas + 1 (solo = 2). El +1 es la válvula para cuando algo se traba. Si nunca lo alcanzas, está demasiado alto

Trazabilidad: del requerimiento al deploy (y de vuelta)



- **Dos preguntas espejo:**
 - "¿POR QUÉ existe este cambio?" (del código al requerimiento)
 - "¿Esto ya está implementado? ¿En qué commit?" (del requerimiento al código)
- **El mecanismo es humilde: referenciar desde el PR el issue que lo originó — Closes #12 (GitHub) • Fixes AB#12 (Azure)**
- Al mergear: la plataforma cierra el issue y deja el enlace permanente en AMBAS direcciones (ojo: AB#12 a secas SÓLO enlaza, no cambia el estado)
- En industrias reguladas es obligatorio; en cualquier equipo serio, convierte 'creo que está en prod' en 'está desde el build #142'

GitHub Projects: cómo se arma (riel canónico)

Concepto	En GitHub Projects (v2)
Todo es un issue	Historias, tareas y bugs son issues con labels (story/task/bug)
Jerarquía	Sub-issues: la épica tiene historias como sub-issues; cada historia, sus tareas (barra de progreso automática)
Sprint	Plazo fijo CON un objetivo. El campo Iteration da el plazo; el objetivo lo escribís vos (la herramienta no tiene dónde)
Board	Vista Board por campo Status (Todo / In Progress / Done)
Automatización	Workflows del Project: 'issue cerrado → Done'
CLI	gh project create · gh issue create · gh project item-add

- **Filosofía: flexible y cercana al código — poca estructura impuesta · gratis, sin tarjeta, en tu cuenta (demo en un rato)**

Azure Boards: la versión estructurada (riel Azure)

Concepto	GitHub Projects	Azure Boards
Jerarquía	Sub-issues + labels (convención)	Work items NATIVOS: Epic → User Story → Task
Criterios de aceptación	En el body del issue	Campo propio del work item
Sprint	Campo Iteration	Iterations del proyecto + vista Sprints con capacity
Board	Vista por Status	Kanban board por nivel de backlog
Trazabilidad	Closes #N	Fixes AB#N en la DESCRIPCIÓN del PR (AB#N solo linkea) — y policy que EXIGE work item por PR
Queries	Filtros de la vista	Lenguaje de queries propio (WIQL) + charts

- Estructura de fábrica (nota: el proceso Agile intercala Feature entre Epic y Story — tu 'épica' del TP puede ser una Feature) • stack del examen AZ-400 • gratis hasta 5 usuarios sin tarjeta

De la épica al PR que cierra el issue: el recorrido

- **Todo esto, paso a paso y en pantalla, está en el video del TP3:**
 - 1. Crear el Project desde la web + las etiquetas epic / story / task
 - 2. La épica + una historia bien escrita (formato + criterios) — y una MAL escrita, para diagnosticarla
 - 3. Jerarquía con sub-issues: la historia cuelga de la épica; las tareas, de la historia
 - 4. Campo Iteration + board por Status + límite de trabajo en progreso
 - 5. El momento clave: rama → cambio → PR con 'Closes #N' → merge → el issue se cierra SOLO y el board se mueve SOLO
 - 6. Navegar la trazabilidad: del issue cerrado al PR y a los commits
- **Mirálo con tu proyecto abierto al lado y hacelo en paralelo — se pausa las veces que haga falta**

TP3 — Planificación: qué tenés que hacer

- **Escenario: el cliente quiere visibilidad — montás la gestión del proyecto sobre TU repositorio**
- 1 • Estructura: la épica + la historia (formato completo + criterios) + sus 2 tareas + el bug, con las etiquetas epic/story/task — jerarquía NAVEGABLE
- 2 • Sprint con la duración que elijas (y justificues) + historias asignadas + board con automatización mínima + límite de WIP
- 3 • decisiones.md — las decisiones que tomaste (el detalle, en la filmina que sigue)
- 4 • Trazabilidad: 1 PR mergeado (protecciones del TP1, approvals en 0) que cierra su issue — desde la TAREA cerrada se navega al commit y de ahí a la épica

Ojo: acá lo que hace la guía ES la entrega

	Lo que hace la guía/video	Lo que entregás
TP2	Un EJEMPLO (contenerizar la app de la cátedra)	Tu propia app contenerizada
TP3	La épica, la historia, las tareas, el bug, el tablero, el PR	ESO MISMO, en tu repo

- **No hay una segunda lista de cosas que tengas que inventar: reproducís lo del video sobre tu repo**
- **Lo ÚNICO que decidís vos (y es lo que se defiende): la duración del sprint • el límite de trabajo en progreso • el diagnóstico de la historia mal escrita • los problemas que tuviste**
- Copiar el procedimiento está bien. No poder explicarlo, no: la defensa pregunta POR QUÉ cada criterio se puede verificar y POR QUÉ esos dos números
- **Lo que no hace falta entregar (si te queda en el tablero, no molesta): la historia mal escrita — de ésta sí va el diagnóstico —, la columna Review, el Roadmap, el responsable y el reporte de métricas**

TP3 — Entregables, defensa y evaluación

Entregable	Contenido
URL repo + URL del Project (PÚBLICO)	Con Azure Boards: capturas completas o acceso de lectura
decisiones.md	Las 5 líneas que escribís vos (detalladas en la filmina anterior)

- **1 PR mergeado: implementa una de las dos tareas de la historia y la cierra solo con Closes #N**
- SIN evidencias.md: tu proyecto es público y se ve solo (con Azure Boards sí van capturas)
- **Defensa (en P1, clase 5): 25% técnico • 25% decisiones.md • 50% defensa oral — el TP3 pesa 10% de P1**
- Preguntas típicas: ¿épica vs historia vs tarea con TU ejemplo? • ¿qué hace buena a una historia? • navegá de una tarea cerrada al commit y de ahí a la épica • ¿por qué ese límite de WIP? • ¿qué te da esto que un Trello no?

Cierre — para la próxima clase

1

Armá tu proyecto esta semana

Reproduciendo el video: épica + historia con criterios + 2 tareas + bug

2

TP2 avanzando

App elegida y dockerizándose — la próxima clase el pipeline la va a necesitar

3

Próxima clase: CI — Pipelines as Code

El pipeline llega a tus PRs: build + tests automáticos en cada cambio

4

Se acerca P1 (clase 5)

TPs 1-4 se defienden juntos — calendario en mano, no acumules